Python: Cheat Sheet

Variables & Basic Types

```
x = 5            # int
y = 3.14         # float
name = "Alice"   # str
is_ok = True     # bool
nothing = None   # NoneType

type(x)          # check type -> <class 'int'>
int("10")        # convert to int
str(10)          # convert to str
float("2.5")     # convert to float

# Variables can be reassigned freely...
x = 10           # now 10
x = x + 5        # now 15

# ...even to a completely different type (dynamic typing)
x = "now a string"   # perfectly valid
x = [1, 2, 3]        # and now it's a list

# type() reveals any value's class. (It's a built-in, not an operator —
# in fact type is itself a class: type(x) returns the class of x.)
type(3)              # <class 'int'>
type(3.14)           # <class 'float'>
type("x")            # <class 'str'>
type(True)           # <class 'bool'>
type(None)           # <class 'NoneType'>
type([1])            # <class 'list'>
type((1,))           # <class 'tuple'>   (the comma makes the tuple)
type({1})            # <class 'set'>     ({} braces with no colon -> set)
type({"a": 1})       # <class 'dict'>
type(b"x")           # <class 'bytes'>

# It works on class instances (and on classes) too
class Dog:
    pass

rex = Dog()
type(rex)            # <class '__main__.Dog'>  (qualified by module name)
type(rex) is Dog     # True
type(rex).__name__   # "Dog"
type(Dog)            # <class 'type'>  -> a class is itself an object
```

Printing

```
print("Hello")              # Hello
print("Hello", "world")     # Hello world  (items joined by a space)

# Mixing types — print handles the conversion
age = 30
print("Age:", age)          # Age: 30
```

```
# With an f-string
print(f"Age is {age}")      # Age is 30

# sep: change the separator between items
print("a", "b", "c", sep="-")    # a-b-c

# end: change what's printed at the end (default is a newline)
print("no newline", end=" ")
print("same line")          # no newline same line
```

Strings

```
s = "hello world"     # double or single quotes — interchangeable
s = 'hello world'
s = 'She said "hi"'   # use one kind to embed the other without escaping
s = """multi
line"""               # triple quotes span multiple lines

# No separate char type: a single character is just a string of length 1
c = "hello"[0]        # "h" -> still a str, not a char
type(c)               # <class 'str'>

s = "hello world"

len(s)               # 11
s.upper()            # "HELLO WORLD"
s.lower()            # "hello world"
s.title()            # "Hello World"
s.replace("l", "L")  # "heLLo worLd"
s.split(" ")         # ['hello', 'world']
"-".join(["a", "b"]) # "a-b"
s.strip()            # remove surrounding whitespace
s.startswith("he")   # True
s.endswith("ld")     # True
s.find("o")          # 4   -> index of first match (-1 if not found)
s.count("l")         # 3   -> number of occurrences
"42".zfill(5)        # "00042"  -> pad with leading zeros
"Hi {}".format(s)    # "Hi hello world"  (older alternative to f-strings)
s[0]                 # "h"   (first char)
s[-1]                # "d"   (last char)
s[0:5]               # "hello" (slice)

# Strings are IMMUTABLE (read-only) — you can't change them in place
# s[0] = "H"         # TypeError!
s = "H" + s[1:]      # instead, build a NEW string

# f-strings (formatting)
name = "Alice"
age = 30
f"{name} is {age}"   # "Alice is 30"

# Format specs: add ":" inside the braces
x = 3.14159
n = 1234567
f"{x:.2f}"           # "3.14"         2 decimal places
f"{n:,}"             # "1,234,567"    thousands separator
f"{x:.1%}"           # "314.2%"       percentage
f"{'hi':<8}|"        # "hi      |"    left-align in width 8
f"{'hi':>8}|"        # "      hi|"    right-align
f"{'hi':^8}|"        # "   hi   |"    center
```

```
f"{x=}"              # "x=3.14159"    debug form: shows name and value
```

Bytes

Text (`str`) and raw bytes (`bytes`) are different types: a string holds characters, bytes hold numbers 0–255. Converting between them always goes through an encoding (UTF-8 unless you have a reason otherwise).

```
data = b"hello"            # bytes literal: b-prefix, raw bytes, NOT text
type(data)                 # <class 'bytes'>

# str -> bytes: encode
b = "città".encode("utf-8")     # b'citt\xc3\xa0'
len(b)                          # 6 -> "à" takes TWO bytes in UTF-8

# bytes -> str: decode
b.decode("utf-8")          # "città"

# Indexing bytes yields INTEGERS, not 1-char strings
b"hi"[0]                   # 104  -> the byte value of "h"
bytes([104, 105])          # b'hi' -> and back from a list of ints

# str and bytes never mix silently
# "a" + b"b"               # TypeError: can only concatenate str to str
```

Lists

```
nums = [1, 2, 3]

nums.append(4)        # [1, 2, 3, 4]
nums.insert(0, 0)     # [0, 1, 2, 3, 4]
nums.pop()            # remove & return last
nums.remove(2)        # remove first matching value
nums[0]               # first item
nums[-1]              # last item
nums[1:3]             # slice
len(nums)             # length
sorted(nums)          # new sorted list
nums.reverse()        # reverse in place
3 in nums             # membership test -> True/False

# Elements don't have to share a type — any mix is fine (heterogeneous)
mixed = [1, "two", 3.0, True, [4, 5]]    # even other lists

# Indexing past the end raises (reads never auto-grow the list)
# nums[99]            # IndexError: list index out of range
```

Slicing

`sequence[start:stop:step]` — the `stop` index is excluded. Works on strings, lists, and tuples.

```
s = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

s[2:5]        # [2, 3, 4]          start:stop
s[:3]         # [0, 1, 2]          from the beginning
s[7:]         # [7, 8, 9]          to the end
```

```
s[-3:]        # [7, 8, 9]          last three
s[:-2]        # [0, 1, 2, 3, 4, 5, 6, 7]   all but the last two
s[::2]        # [0, 2, 4, 6, 8]    every 2nd item (step)
s[1::2]       # [1, 3, 5, 7, 9]    every 2nd, starting at index 1
s[::-1]       # [9, 8, 7, ... 0]   reversed (step of -1)

"hello"[::-1] # "olleh"            same syntax on strings

# Out-of-range slices are forgiving (unlike indexing): they clamp
s[2:100]      # [2, 3, 4, 5, 6, 7, 8, 9]   no IndexError, stops at the end

# Slice assignment replaces a range (lists only)
s[0:2] = [100]    # [100, 2, 3, 4, 5, 6, 7, 8, 9]
```

Dictionaries

```
person = {"name": "Alice", "age": 30}

person["name"]              # "Alice"
# person["email"]          # KeyError: a missing key raises on [] access
person.get("email")         # None    -> no error; None when key is missing
person.get("email", "n/a")  # "n/a"   -> supply your own default
person["email"] = "a@x.com" # add / update
person.update({"age": 31})  # merge another dict in place
del person["age"]           # delete a key
person.pop("email")         # remove a key and return its value
# Deleting a MISSING key raises, just like reading one:
# del person["email"]       # KeyError
# person.pop("email")       # KeyError
person.pop("email", "n/a")  # "n/a" -> pop with a default never raises
"name" in person            # check key exists -> True

# setdefault: return the value if the key exists; otherwise INSERT it
# with the given default and return that. It can MUTATE the dict.
d = {"a": 1}
d.setdefault("a", 99)   # 1   -> "a" exists: returned as-is, d unchanged
d.setdefault("b", 99)   # 99  -> "b" missing: d is now {"a": 1, "b": 99}

# keys(), values(), items() return live VIEW objects, not lists.
# Views are iterable and reflect later changes to the dict.
scores = {"x": 1, "y": 2}
ks = scores.keys()          # dict_keys(['x', 'y'])
scores["z"] = 3
list(ks)                    # ['x', 'y', 'z']  -> the view saw the change
for key, val in scores.items():   # idiomatic way to iterate key/value pairs
    print(key, val)
list(scores.values())       # [1, 2, 3]  -> wrap in list() for a snapshot

# Merge into a NEW dict with | (Python 3.9+)
combined = {"a": 1} | {"b": 2}    # {"a": 1, "b": 2}
```

Tuples & Sets

```
point = (3, 4)        # tuple (immutable)
x, y = point          # unpacking
point[0]              # 3    -> index like a list
# point[5]            # IndexError: tuple index out of range
```

```
# Tuples mix types too — the classic use: a fixed-shape record
user = ("Alice", 30, True)    # name, age, active

s = {1, 2, 2, 3}      # set -> {1, 2, 3} (unique)
s.add(4)
s.union({5, 6})
s.intersection({2, 3})

# Convert between them
nums = [1, 2, 2, 3, 3]
unique = set(nums)        # list -> set: {1, 2, 3}  (drops duplicates)
back = list(unique)       # set -> list: [1, 2, 3]

# Common idiom: remove duplicates from a list
deduped = list(set(nums)) # [1, 2, 3]  (note: order is NOT preserved)
```

Operators

```
# Arithmetic operators
7 + 2        # 9
7 - 2        # 5
7 * 2        # 14
7 / 2        # 3.5    true division ALWAYS returns a float
7 // 2       # 3      floor division (rounds DOWN to a whole number)
7 % 2        # 1      modulo (the remainder)
7 ** 2       # 49     exponentiation (power)
-7 // 2      # -4     floor rounds toward -infinity, NOT toward zero
divmod(7, 2) # (3, 1) quotient and remainder together

# Comparison (relational) operators -> always return a bool
2 == 2       # True    equal value
2 != 3       # True    not equal
3 > 2        # True    also <, >=, <=
# Comparisons can be chained, like in maths — most languages can't do
# this (in Java/C, 1 < x < 10 means (1 < x) < 10: a bool vs a number)
x = 5
1 < x < 10   # True    same as: 1 < x and x < 10

# Logical operators
True and False    # False   both sides must be truthy
True or False     # True    at least one side truthy
not True          # False   negation

# GOTCHA: and/or return one of the OPERANDS, not a strict True/False,
# and short-circuit (stop as soon as the result is known).
"a" and "b"       # "b"          first is truthy -> yields the second
0 or "fallback"   # "fallback"   first is falsy  -> yields the second
"" or "default"   # "default"    common way to supply a default value

# Identity vs equality: == asks "equal VALUE?", is asks "the SAME object?"
# Every object lives at one identity (see id()); `is` returns True only
# when both sides are literally that one object, never comparing contents.
a = [1, 2]
b = [1, 2]      # a second, separate list that happens to have equal contents
c = a           # an alias: c and a are two names for the SAME list
a == b          # True    equal contents (value comparison)
a is b          # False   two distinct objects, however equal
a is c          # True    one object, two names

# id() returns that identity: an int unique to the object for its whole
```

```
# lifetime (in CPython it's the memory address). `is` compares ids.
# Identity is assigned by the interpreter, not the class: there is no
# dunder to implement or override for it (unlike ==, which calls __eq__).
id(a)           # e.g. 4344961088 — some int; different on every run
id(a) == id(c)  # True    what `is` actually checks: the same identity
id(a) == id(b)  # False   equal lists, but two different objects

# Use `is` ONLY for singletons like None — never to compare values.
# (Caching of small ints/strings can make `is` LOOK like == by accident.)
x = None
x is None       # True    the correct way to test for None (not ==)

# WHY not ==? Because == calls __eq__, and a class can override it to
# answer anything — so `x == None` can lie. Identity can't be fooled,
# and since None is a SINGLETON (one object ever), `is` asks exactly
# the right question: "is this that one object?"
class Weird:
    def __eq__(self, other):
        return True          # claims equality with EVERYTHING

w = Weird()
w == None       # True    -> the lie: __eq__ decided the answer
w is None       # False   -> the truth: w is not the None object
```

Mutability & References

Python has no value-type vs reference-type split (unlike Java/C#): **every** variable holds a reference to an object. What actually matters is whether the object is **mutable** (list, dict, set) or **immutable** (int, float, bool, str, bytes, tuple).

```
# Assignment NEVER copies — it gives the SAME object another name
a = [1, 2, 3]
b = a                 # b is an ALIAS of a, not a copy
b.append(4)
a                     # [1, 2, 3, 4] -> the change shows through a too
a is b                # True   (one object, two names)

# "Immutable" means the OBJECT itself can never change after creation.
# There is no way to turn the int 10 into an 11: operations that look
# like changes actually build a NEW object and REBIND the name to it.
x = 10
y = x                 # x and y -> the same int object (the 10)
x is y                # True
y += 1                # no mutation: creates a NEW int 11, rebinds y to it
x                     # 10    -> the 10 object was never touched
x is y                # False -> the names now point to different objects

# Strings work the same way: every "modification" is a new object
s = "hi"
t = s
t += "!"              # t now points to a NEW string "hi!"
s                     # "hi"  -> original untouched (strings can't change)

# So aliasing an immutable object is always harmless — nothing can
# alter it through EITHER name. That's why int/float/bool/str/bytes/
# tuple behave like Java/C# "value types", despite being references.

# Copy a list when you need independence
c = a.copy()          # or a[:] or list(a) -> a NEW list
c.append(99)
```

```
a                      # [1, 2, 3, 4]  -> unaffected

# GOTCHA: those copies are SHALLOW — nested objects are still shared
grid = [[1, 2], [3, 4]]
flat = grid.copy()
flat[0].append(99)
grid                   # [[1, 2, 99], [3, 4]] -> inner list was shared!
# copy.deepcopy (from the copy module) copies every level instead.

# Arguments are passed the same way: the function receives a REFERENCE,
# so mutating a passed-in list changes the caller's list
def add_item(lst):
    lst.append("x")

data = [1]
add_item(data)
data                   # [1, 'x']
```

Conditionals

```
x = 5
if x > 10:
    print("big")
elif x == 10:
    print("ten")
else:
    print("small")

# Ternary
label = "even" if x % 2 == 0 else "odd"

# Walrus := assigns AND returns a value in the same expression (3.8+)
if (n := len("hello")) > 3:
    print(f"length is {n}")     # n is assigned inside the condition
```

Truthy & Falsy Values

In a boolean context (like `if`), values are automatically treated as True or False. `bool(x)` shows what a value converts to.

```
# Falsy values (treated as False):
bool(False)   # False
bool(None)    # False
bool(0)       # False   (also 0.0)
bool("")      # False   (empty string)
bool([])      # False   (empty list, dict, tuple, set)
bool({})      # False

# Everything else is truthy (treated as True):
bool(42)      # True
bool("hi")    # True
bool([0])     # True    (non-empty, even if it contains 0)

# Common idiom: check for emptiness directly
items = []
if not items:
    print("the list is empty")
```

Match / Case

Structural pattern matching (Python 3.10+) — a cleaner alternative to long `if`/`elif` chains.

```
command = "start"

match command:
    case "start":
        print("starting")
    case "stop" | "halt":          # | matches multiple patterns
        print("stopping")
    case _:                        # _ is the default (wildcard)
        print("unknown command")

# Cases are tried TOP TO BOTTOM: the first match wins, runs its body,
# and the rest are skipped — no fall-through, no break (unlike switch).
# ORDER MATTERS with overlapping patterns: put narrow cases first.
match 5:
    case int():                    # broad case listed first...
        print("an int")            # -> wins
    case 5:                        # ...so this never gets a chance
        print("exactly five")

# If NO case matches (and there is no _), nothing runs — no error.
match "no case matches me":
    case 1:
        print("one")               # skipped; the match just does nothing

# It can also destructure data
point = (0, 5)
match point:
    case (0, 0):
        print("origin")
    case (0, y):                   # binds y to the second value
        print(f"on the y-axis at {y}")
    case (x, y):
        print(f"at {x}, {y}")

# GOTCHA: a bare name in a case is a CAPTURE, not a comparison.
# It matches ANYTHING and binds it to that name — it does NOT compare
# against an existing variable called `expected`:
expected = "stop"
match "anything":
    case expected:                 # always matches; expected is rebound!
        print(expected)            # "anything"

# A DOTTED name, however, IS compared as a constant:
class Command:
    STOP = "stop"

match "stop":
    case Command.STOP:             # attribute access -> real comparison
        print("stopping")

# Guard: attach a condition to a pattern with `if`
match (4, 4):
    case (x, y) if x == y:         # pattern must match AND guard be true
        print(f"on the diagonal at {x}")
    case (x, y):
        print("elsewhere")
```

```
# Sequence pattern: destructure a list; * collects the rest
match [1, 2, 3, 4]:
    case [first, *rest]:
        print(first, rest)        # 1 [2, 3, 4]

# Mapping pattern: matches if these KEYS exist (extra keys are fine)
match {"action": "move", "x": 10}:
    case {"action": act}:
        print(act)                # "move"

# Class pattern: match on the type and its attribute values
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

match Point(0, 7):
    case Point(x=0, y=y):         # an instance of Point with x == 0
        print(f"on the y-axis at {y}")
```

Loops

```
nums = [1, 2, 3]
x = 3

for n in nums:
    print(n)

for i in range(5):       # 0,1,2,3,4
    print(i)

for i, val in enumerate(nums):   # index + value
    print(i, val)

for i, val in enumerate(nums, start=1):   # count from 1 instead of 0
    print(i, val)

# zip: iterate two (or more) sequences in PARALLEL
names = ["Alice", "Bob"]
ages = [30, 25]
for name, age in zip(names, ages):
    print(name, age)     # Alice 30 / Bob 25  (stops at the shorter one)

# reversed: iterate backwards (works on lists, tuples, ranges, strings)
for n in reversed(nums):
    print(n)             # 3, 2, 1

# Iterating a dict yields its KEYS; use .items() for key/value pairs
person = {"name": "Alice", "age": 30}
for key in person:
    print(key, person[key])
for key, val in person.items():      # see Dictionaries
    print(key, val)

while x > 0:
    x -= 1

# Python has NO do-while; the idiom is `while True` + a break inside
count = 0
while True:
```

```
    count += 1
    if count == 3:
        break            # the exit condition sits inside the body

# break    -> exits the loop early
# continue -> skips to the next iteration

# break / continue in action
for n in range(10):
    if n == 3:
        continue     # skip 3, go to next iteration
    if n == 6:
        break        # stop the loop entirely
    print(n)         # prints 0, 1, 2, 4, 5

# Loops have an `else` (NOT a `finally`):
# it runs only if the loop finished WITHOUT hitting a break.
for n in nums:
    if n < 0:
        print("found a negative")
        break
else:
    print("no negatives found")   # runs only if no break occurred

# GOTCHA: never modify a list WHILE looping over it — removals shift
# the elements and the loop silently SKIPS some:
doomed = [1, 2, 3, 4]
for n in doomed:
    doomed.remove(n)     # tries to empty the list...
doomed                   # [2, 4]  -> half survived!

# Fix: loop over a COPY, or build a new list with a comprehension
items = [1, 2, 3, 4]
for n in items[:]:       # [:] makes a copy to iterate
    if n % 2 == 0:
        items.remove(n)
items                    # [1, 3]
evens_removed = [n for n in [1, 2, 3, 4] if n % 2 != 0]   # [1, 3]
```

Range

`range` produces a sequence of integers lazily (it doesn't build a list).

```
range(5)            # 0,1,2,3,4       -> stop only
range(2, 6)         # 2,3,4,5         -> start, stop (stop excluded)
range(0, 10, 2)     # 0,2,4,6,8       -> start, stop, step
range(5, 0, -1)     # 5,4,3,2,1       -> negative step counts down

list(range(5))      # [0,1,2,3,4]     -> materialize into a list
len(range(5))       # 5
3 in range(5)       # True

for i in range(3):  # most common use: repeat / index
    print(i)
```

Comprehensions

```
nums = [1, 2, 3, 4]
```

```python
# List comprehension -> [...]: build a new list from an iterable
squares = [n**2 for n in range(5)]        # [0, 1, 4, 9, 16]
# List comprehension with a filter (the trailing `if`)
evens   = [n for n in nums if n % 2 == 0] # [2, 4]  -> keeps only evens
# Dict comprehension -> {key: value ...}: build a new dict
lookup  = {n: n**2 for n in range(3)}     # {0: 0, 1: 1, 2: 4}
# Set comprehension -> {...}: build a new set (unique values)
sizes   = {len(w) for w in ["hi", "ok", "hey"]}  # {2, 3}

# There is NO tuple comprehension: (n for n in nums) is a GENERATOR
# expression, not a tuple. Build a tuple explicitly with tuple(...):
nums_t  = tuple(n * 2 for n in nums)      # (2, 4, 6, 8)
```

Generators

Generators produce values one at a time, on demand, instead of building a whole list in memory — efficient for large or streaming data.

```python
# Generator expression: like a comprehension, but with ()
gen = (n**2 for n in range(5))   # nothing computed yet
next(gen)                        # 0   (produces one value at a time)
next(gen)                        # 1
list(gen)                        # [4, 9, 16]  (the rest)

# Generator function: uses `yield` to emit values lazily
def countdown(n):
    while n > 0:
        yield n                  # pauses here, resumes on next request
        n -= 1

for x in countdown(3):           # 3, 2, 1
    print(x)

# A for loop consumes any iterator; next() steps through one manually.
# Built-ins like range(), enumerate(), zip() are lazy in the same way.
```

Iterators vs generators:

- An **iterator** is any object with `__iter__()` and `__next__()` methods (the "iterator protocol"). Every `for` loop drives one behind the scenes.
- A **generator** is the easy way to *make* an iterator — via a generator function (`yield`) or a generator expression `(...)`. All generators are iterators; not all iterators are generators.

```python
nums = [10, 20, 30]
it = iter(nums)      # get an iterator from an iterable
next(it)             # 10
next(it)             # 20
# next() past the end raises StopIteration

# The manual (class) way to build an iterator — what generators save you from:
class Counter:
    def __init__(self, limit):
        self.n, self.limit = 0, limit
    def __iter__(self):
        return self                  # an iterator returns itself
    def __next__(self):
        if self.n >= self.limit:
```

```
            raise StopIteration      # signals "no more values"
        self.n += 1
        return self.n

list(Counter(3))     # [1, 2, 3]
```

Functions

Defining & Calling

```
# Takes no arguments, returns nothing (implicitly returns None)
def say_hello():
    print("hello")

say_hello()                 # prints "hello"
result = say_hello()        # result is None (no return statement)

# Returns a value with `return`
def add(a, b):
    return a + b

add(2, 3)                   # 5

# Empty function placeholder (pass = do nothing)
def todo_later():
    pass
```

Docstrings

```
# A docstring is a string literal (triple-quoted by convention) placed as
# the FIRST statement of a function, class, or module. Python stores it
# as the object's documentation — it is what help() and editors show.
def area(width, height):
    """Return the area of a rectangle."""
    return width * height

class Dog:
    """A pet that can bark.

    Docstrings can span multiple lines: a one-line summary first,
    then details after a blank line.
    """

area.__doc__      # 'Return the area of a rectangle.'
Dog.__doc__       # 'A pet that can bark.\n\n    Docstrings can span...'
# help(area)      # pretty-prints the docstring
```

Define Before You Call

A name must exist by the time the line that uses it **runs** — not by where it sits in the file.

```
# ping()           # NameError: 'ping' is not defined yet
def ping():
    return "pong"
ping()              # works now that ping is defined
```

```
# A function may reference another defined LATER, as long as both exist
# before the top-level call actually runs:
def main():
    return helper()      # 'helper' isn't looked up until main() is called
def helper():
    return "ready"
main()                   # "ready" -> both defined before this line runs
```

Default Values

```
def greet(name, greeting="Hi"):   # greeting has a default value
    return f"{greeting}, {name}!"

greet("Alice")              # "Hi, Alice!"  -> uses the default greeting
greet("Bob", "Hello")       # "Hello, Bob!" -> default is overridden
greet("Eve", greeting="Hey")# "Hey, Eve!"   -> override by keyword name

# GOTCHA: a default value is created ONCE (at definition), not per call.
# A mutable default (list/dict) is then SHARED across calls, like a
# static variable — it accumulates instead of resetting:
def add_item(item, items=[]):
    items.append(item)
    return items

add_item("a")    # ['a']
add_item("b")    # ['a', 'b']   <- same list reused, NOT a fresh one!

# Fix: default to None and create the object inside the function
def add_item(item, items=None):
    if items is None:
        items = []           # fresh list every call
    items.append(item)
    return items

add_item("a")    # ['a']
add_item("b")    # ['b']        <- independent now
```

Positional vs Keyword Arguments

```
def make_user(name, age, city):
    return f"{name}, {age}, {city}"

# Positional: matched by ORDER
make_user("Alice", 30, "Rome")

# Keyword (named): matched by NAME, so order doesn't matter
make_user(name="Alice", city="Rome", age=30)

# Mixed: positional args must come BEFORE keyword args
make_user("Alice", city="Rome", age=30)   # ok
# make_user(name="Alice", 30, "Rome")     # SyntaxError
```

Variable-length Parameters

```
# *args -> extra POSITIONAL args collected into a tuple
def total(*args):
    return sum(args)
```

```
total(1, 2, 3)        # 6   -> args is (1, 2, 3)

# **kwargs -> extra KEYWORD args collected into a dict
def show(**kwargs):
    return kwargs

show(name="Alice", age=30)   # {'name': 'Alice', 'age': 30}

# Unpacking when CALLING: * spreads a list, ** spreads a dict
nums = [1, 2, 3]
total(*nums)          # 6   -> same as total(1, 2, 3)
data = {"name": "Bob", "age": 25}
show(**data)          # {'name': 'Bob', 'age': 25}

# All four kinds together — ORDER MATTERS, must be:
#   1. regular (required)   2. default   3. *args   4. **kwargs
def f(a, b=2, *args, **kwargs):
    print(a, b, args, kwargs)

f(1)                        # 1 2 () {}
f(1, 9)                     # 1 9 () {}
f(1, 9, 10, 11, x=5)        # 1 9 (10, 11) {'x': 5}
# def f(*args, a): ...      # a after *args = keyword-ONLY argument
# def f(a, b=2, c): ...     # SyntaxError: non-default after default
```

Returning Multiple Values

```
def min_max(nums):
    return min(nums), max(nums)    # returns a tuple

lo, hi = min_max([3, 1, 5])        # unpack the result -> lo=1, hi=5
```

Lambdas

```
# A lambda is a small anonymous function (a single expression)
double = lambda x: x * 2
double(5)             # 10

# Often used inline, e.g. as a sort key
sorted([-3, 1, -2], key=lambda n: abs(n))   # [1, -2, -3]
```

No Overloading

```
# Python has NO function overloading. Defining the same name twice does
# not create two versions — the second definition REPLACES the first.
def area(side):
    return side * side

def area(width, height):     # same name -> this REPLACES the version above
    return width * height

# area(5)             # TypeError: missing argument 'height' (first one is gone)
area(3, 4)            # 12
# For "overloading"-like flexibility, use defaults or *args/**kwargs instead.
```

Nested Functions & Closures

```
# A function can be defined INSIDE another function. The inner one is
# local to the outer and can read the outer's variables (a closure).
def make_counter(start=0):
    count = start
    def increment():
        nonlocal count       # rebind the ENCLOSING variable (not a global)
        count += 1
        return count
    return increment         # return the inner function itself

counter = make_counter()
counter()    # 1
counter()    # 2   -> count persists between calls, captured by the closure

# Reading an enclosing variable needs nothing special; only REBINDING it
# requires `nonlocal` (otherwise the assignment makes a new local instead).
# The same works inside a method: a def there is just a local helper.

# Scope rules (LEGB): a name is looked up Local -> Enclosing -> Global ->
# Built-in. Use `nonlocal x` to rebind an enclosing variable, and
# `global x` to rebind a top-level (module) variable from inside a function.
count = 0
def bump():
    global count        # without this, `count = ...` would make a new local
    count += 1
bump()
count                   # 1
```

Decorators

A decorator is a function that wraps another function to add behavior, without changing the original. `@name` above a function applies it.

```
def shout(func):
    def wrapper(*args, **kwargs):     # *args/**kwargs pass through anything
        result = func(*args, **kwargs)
        return result.upper()
    return wrapper

@shout                                # same as: greet = shout(greet)
def greet(name):
    return f"hi {name}"

greet("alice")        # "HI ALICE"  -> wrapped by shout

# Common built-in decorators you'll see:
#   @staticmethod / @classmethod  (in classes, shown below)
#   @property                     (expose a method like an attribute)
#   @functools.cache              (cache a function's results)
```

Classes

```
class Dog:
    # Class attribute (shared by all instances)
```

```
    species = "Canis familiaris"

    # Constructor: runs when you create an instance
    def __init__(self, name, age):
        self.name = name      # instance attributes
        self.age = age

    # Method
    def bark(self):
        return f"{self.name} says woof!"

    # String representation (used by print)
    def __repr__(self):
        return f"Dog({self.name}, {self.age})"

# Create instances
rex = Dog("Rex", 3)
rex.name          # "Rex"
rex.bark()        # "Rex says woof!"
rex.species       # "Canis familiaris"

# Class attributes work on the CLASS and on INSTANCES
Dog.species       # "Canis familiaris"  -> on the class
rex.species       # "Canis familiaris"  -> on the instance

# Instance attributes only exist on instances, not the class
# Dog.name        # AttributeError — only set inside __init__

# Assigning via an instance does NOT change the class attribute;
# it creates a new instance attribute that shadows it
rex.species = "Dog"   # affects rex only
Dog.species           # still "Canis familiaris"
Dog.species = "X"     # THIS changes it for the class & all instances

# Inheritance
class Puppy(Dog):
    def bark(self):                  # override a method
        return f"{self.name} yips!"

    def info(self):
        base = super().bark()        # call parent's version
        return base

# Multiple inheritance: a class can have more than one parent
class Swimmer:
    def move(self):
        return "swimming"

class Walker:
    def move(self):
        return "walking"

class Amphibian(Walker, Swimmer):    # inherits from both
    pass

Amphibian().move()    # "walking" -> parents checked left-to-right (MRO)
Amphibian.__mro__     # the Method Resolution Order Python follows

# Empty class placeholder (pass = do nothing)
class Empty:
    pass
```

Nested Classes

```
# Yes: a class can be defined inside another class. The inner class is
# just an attribute of the outer one (accessed as Outer.Inner) and gets
# NO special access to the outer class or its instances.
class Outer:
    class Inner:              # a nested type, namespaced under Outer
        def hello(self):
            return "hi from Inner"

Outer.Inner                  # the nested class object
inner = Outer.Inner()        # instantiate via the outer's namespace
inner.hello()                # "hi from Inner"
```

Instance, Class & Static Methods

Three kinds of methods, differing in what (if anything) they receive automatically as the first argument:

```
class Pizza:
    count = 0                      # class attribute (shared by all)

    def __init__(self, size):
        self.size = size           # instance attribute
        Pizza.count += 1           # update the shared class attribute

    # Instance method: gets the INSTANCE as `self`.
    # Use when you need the object's own data.
    def describe(self):
        return f"A {self.size} pizza"

    # Class method: gets the CLASS as `cls`. Marked with @classmethod.
    # Use to work with class-level data or build alternate constructors.
    @classmethod
    def how_many(cls):
        return cls.count           # reads the shared class attribute

    # Static method: gets NOTHING automatic. Marked with @staticmethod.
    # Just a plain function grouped inside the class for organization.
    @staticmethod
    def is_valid_size(size):
        return size in ("small", "medium", "large")

p1 = Pizza("large")
p2 = Pizza("small")
p1.describe()                 # "A large pizza"      (needs the instance)
Pizza.how_many()              # 2                     (works via the class)
Pizza.is_valid_size("xl")     # False                (no self/cls needed)
```

Method type	First arg	Decorator	Accesses
Instance	`self`	(none)	instance + class data
Class	`cls`	`@classmethod`	class data only
Static	(none)	`@staticmethod`	neither (self-contained)

Duck Typing

Python doesn't care about an object's type — only whether it has the method/attribute you use. "If it quacks like a duck, treat it as a duck."

```
class Duck:
    def quack(self):
        return "Quack!"

class Person:
    def quack(self):
        return "I'm imitating a duck!"

def make_it_quack(thing):
    return thing.quack()      # no type check — just calls the method

make_it_quack(Duck())      # "Quack!"
make_it_quack(Person())    # "I'm imitating a duck!"
# Works for ANY object that has a .quack() method,
# regardless of its class. Missing it -> AttributeError.
```

You already rely on this everywhere: `len(x)` works on any object with a `__len__`, and a `for` loop works on anything iterable — behavior matters, not type.

Special (Dunder) Methods

"Dunder" (double-underscore) methods let your objects work with built-in operations like `str()`, `int()`, `==`, and `<`. Python calls them for you.

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    # --- conversion ---
    def __str__(self):          # used by str() and print()
        return f"${self.amount}"
    def __int__(self):          # used by int()
        return int(self.amount)
    def __bool__(self):         # used by bool() / if checks
        return self.amount != 0

    # --- comparison ---
    def __eq__(self, other):    # used by ==
        return self.amount == other.amount
    def __lt__(self, other):    # used by <  (and enables sorting)
        return self.amount < other.amount

a = Money(5)
b = Money(10)

str(a)            # "$5"
int(a)            # 5
bool(Money(0))    # False
a == Money(5)     # True
a < b             # True
sorted([b, a])    # sorts to [Money(5), Money(10)] thanks to __lt__
```

Other common ones: `__repr__` (debug representation), `__len__` (`len()`), `__getitem__` (`obj[key]`), and `__gt__` / `__le__` / `__ge__` for the remaining comparisons.

Common Built-ins

```
print("hi")           # output (see Printing section below)
# input("Name: ")     # read user input -> ALWAYS returns a str
# age = int(input("Age: "))   # convert when you need a number
len([1, 2, 3])        # 3
range(0, 5)           # 0,1,2,3,4 (lazy sequence)
sum([1, 2, 3])        # 6
min([3, 1, 2])        # 1   (max([3, 1, 2]) -> 3)
abs(-5)               # 5
round(3.14159, 2)     # 3.14
sorted([3, 1, 2])     # [1, 2, 3]
list(zip([1, 2], ["a", "b"]))     # [(1, 'a'), (2, 'b')]
list(map(str, [1, 2, 3]))         # ['1', '2', '3']  (apply fn to each)
list(filter(lambda n: n > 1, [1, 2, 3]))  # [2, 3]  (keep where True)
```

Error Handling

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Can't divide by zero")
except (TypeError, ValueError) as e:   # catch several types in one clause
    # check which one was actually raised
    if isinstance(e, TypeError):
        print(f"Type problem: {e}")
    else:
        print(f"Value problem: {e}")
except Exception as e:                 # catch-all (put last)
    print(f"Error: {e}")
else:
    print("No errors")
finally:
    print("Always runs")

# Raise an exception yourself
try:
    raise ValueError("must be non-negative")
except ValueError as e:
    print(e)        # "must be non-negative"
```

Defining a Custom Exception

```
# Subclass Exception (or a more specific built-in)
class InsufficientFundsError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError("not enough money")
    return balance - amount

try:
```

```
    withdraw(50, 100)
except InsufficientFundsError as e:
    print(e)        # "not enough money"
```

Files

`with` opens the file as a *context manager*: it automatically closes the file when the block ends — even if an error occurs inside it. This is the recommended way, because forgetting to close a file can lose data or leak resources.

```
# Write (creates / overwrites the file)
with open("file.txt", "w") as f:
    f.write("hello")

# Read
with open("file.txt") as f:
    content = f.read()        # whole file
    # or: lines = f.readlines()

# Append
with open("file.txt", "a") as f:
    f.write("\nmore")
```

Without `with`, you must close the file yourself (use `try`/`finally` so it still closes if something fails):

```
f = open("file.txt")
try:
    content = f.read()
finally:
    f.close()                 # must close manually
```

Imports

```
import math
math.sqrt(16)        # 4.0

from datetime import datetime
datetime.now()

import random as rnd
rnd.randint(1, 10)
```

Defining & Importing Your Own Module

A module is just a `.py` file. Say you create `mymath.py`:

```
# mymath.py
PI = 3.14159

def square(n):
    return n * n

def cube(n):
    return n * n * n
```

Then, from another file in the same folder, you can import it:

```
# main.py
import mymath

mymath.PI            # 3.14159
mymath.square(4)     # 16

# Import specific names
from mymath import square, cube
square(4)            # 16 (no prefix needed)

# Import with an alias
import mymath as mm
mm.cube(3)           # 27

# Runs only when the file is executed directly,
# not when it's imported
if __name__ == "__main__":
    print(square(5))
```

Packages (Folders of Modules)

A **module** is a single `.py` file; a **package** is a folder of modules containing an `__init__.py` file (often empty — it marks the folder as a package and runs on first import):

```
mytools/              # the package
├── __init__.py       # makes the folder importable (can be empty)
└── text.py           # a module inside the package
```

```
# mytools/text.py
def shout(s):
    return s.upper() + "!"
```

```
# Import through the package with dotted names
import mytools.text
mytools.text.shout("hi")        # "HI!"

from mytools.text import shout  # import a specific name
shout("hi")                     # "HI!"

from mytools import text as t   # import the module with an alias
t.shout("hi")                   # "HI!"
```

Package with Subpackages

A package can hold **many modules** and also **other packages** (subpackages, each with its own `__init__.py`), nesting as deep as needed:

```
mytools/
├── __init__.py       # runs ONCE, on first import of mytools
├── text.py
├── numbers.py        # a package holds any number of modules
└── formats/          # a SUBPACKAGE: packages nest
    ├── __init__.py
```

```
    └── csv.py
```

`__init__.py` may be empty, but it can contain any code — it runs once, when the package is first imported. Typical contents: package-level constants, and re-exports that give users a shorter import path:

```
# mytools/__init__.py
# (shown commented: `.text` is a RELATIVE import — the leading dot means
# "from this same package", so the line only runs inside the package)
# from .text import shout    # re-export: lift a name to the package level
# VERSION = "1.0"            # package-level constant
```

```
import mytools
mytools.VERSION                  # "1.0"
mytools.shout("hi")              # "HI!"  -> usable without naming .text,
                                 #           thanks to the re-export

# Reach into a subpackage with the full dotted path
from mytools.formats.csv import to_row
to_row(["a", "b", "c"])          # "a,b,c"

import mytools.numbers
mytools.numbers.double(21)       # 42
```